

SOP: Using ChatGPT to Learn SUSI API & Create Advantech Projects

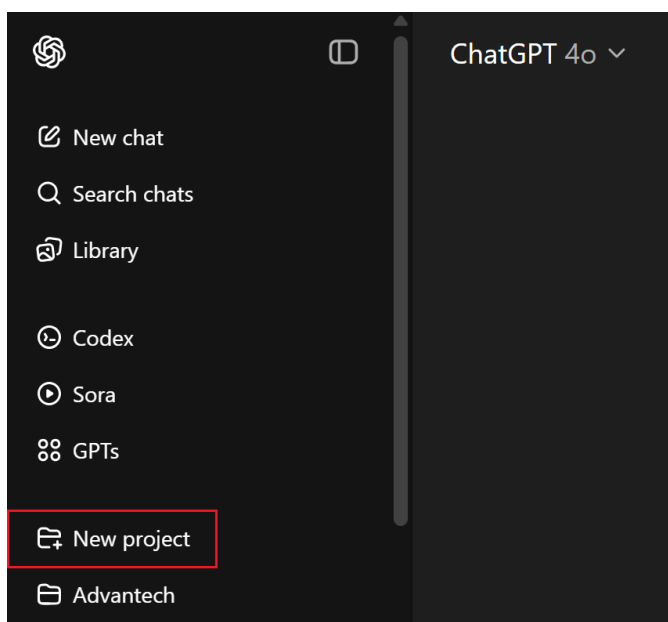
Platform: Advantech EI Series + SUSI 4.0 + AI Use Cases

Tools Needed: ChatGPT Plus, SUSI documentation from GitHub + ESS Wiki, Linux terminal with GCC

Import SUSI Documents into ChatGPT

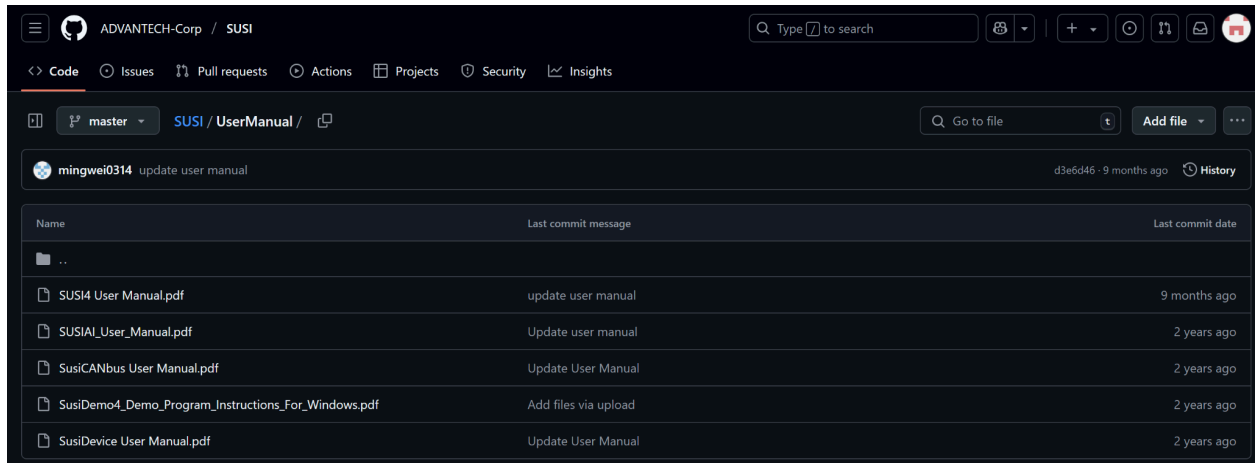
Step-by-step:

- 1.) Open ChatGPT (must have GPT Plus subscription).
- 2.) On the left hand side, click on “**New Project**” and name it whatever you want. In this case, I named my project “**SUSI.**”



- 3.) **Download** the latest documents from the official Advantech Github and websites listed below.

<https://github.com/ADVANTECH-Corp/SUSI/tree/master/UserManual>



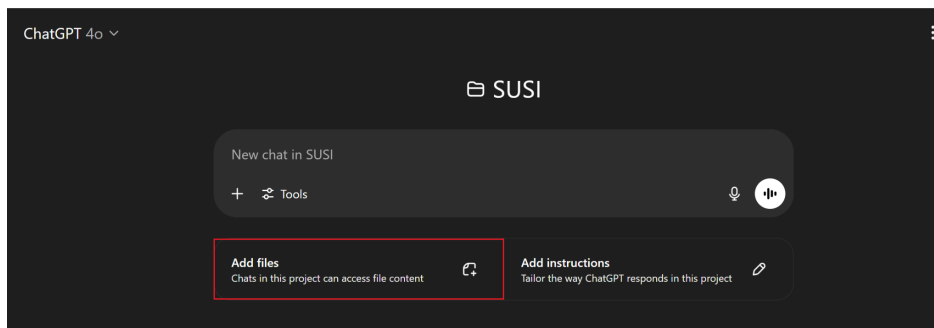
https://ess-wiki.advantech.com.tw/view/SUSI_IoT

Document

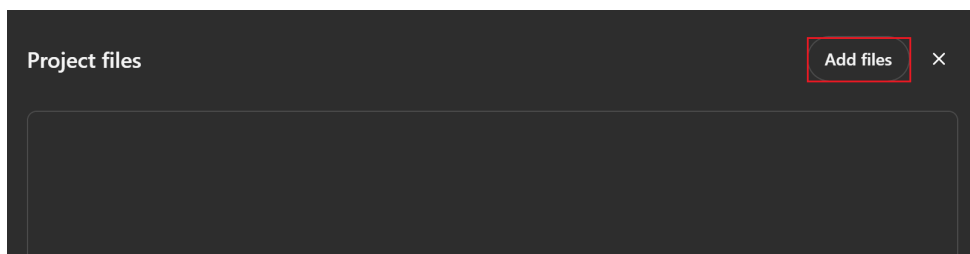
- *User Manual*
- *Demo Program Instructions*
- *Module Porting Guide*

- 4.) Import the documents.

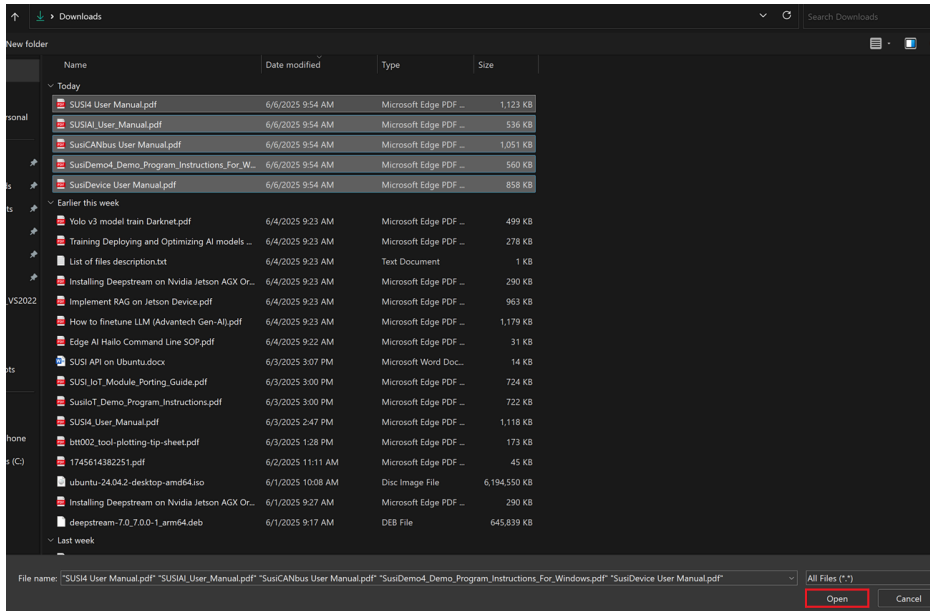
- In your project folder that you created, click on “**Add files.**”



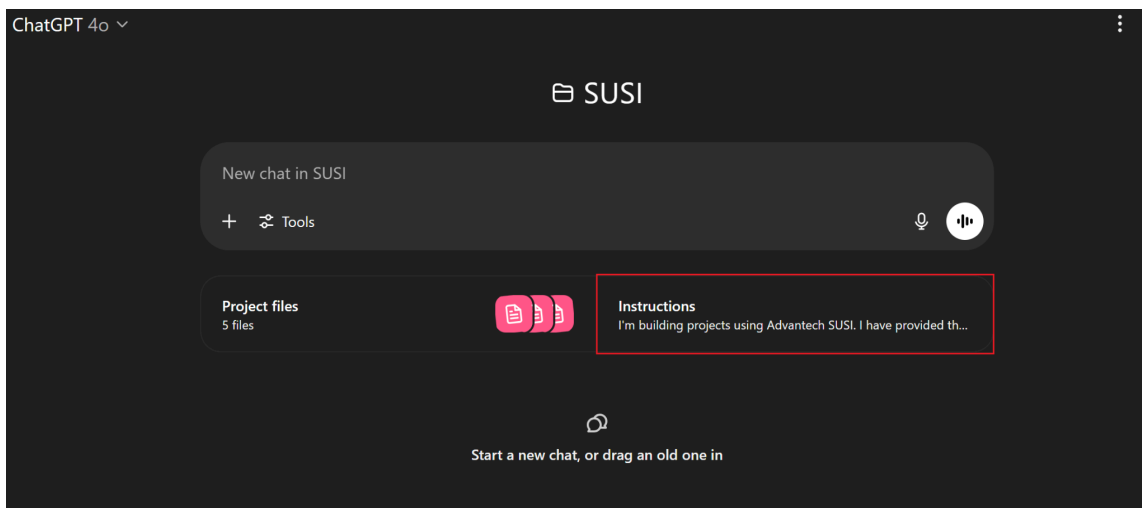
- A new window will open, click on “**Add files.**”



- Locate your files and click on “**Open**”



5.) You can provide GPT instructions to help you with your projects by clicking on “**Instructions**”, adding your desired text, and clicking “**Save**” (e.g. “I’m building projects using Advantech SUSI. I have provided the official documentation, always refer to the documentation. Help me create projects and understand each library, the initialization functions, and how to use them in a real project. “).



Instructions

×

How can ChatGPT best help you with this project?
You can ask ChatGPT to focus on certain topics, or ask it to use a certain tone or format for responses.

I'm building projects using Advantech SUSI. I have provided the official documentation, always refer to the documentation. Help me create projects and understand each library, the initialization functions, and how to use them in a real project.

CancelSave

Outcome: This loads your full SUSI development environment documents into ChatGPT. GPT will now understand and cross-reference SUSI concepts for you.

Learn SUSI by Asking GPT

Here are useful prompts to guide your learning:

- Intro and Understanding:

"What is the purpose of SUSI API? How does it simplify hardware access?"

- Initialization Flow:

"Explain `SusiLibInitialize()` and when to use `SusiLibUninitialize()`."

- Sensor Monitoring:

"Show me a C example that uses `SusiBoardGetValue` to get CPU temperature."

- GPIO:

"How do I configure and toggle GPIO pins using SUSI?"

- JSON Output:

"Explain how `SusiIoTGetMDCapability` returns hardware data as JSON."

- Project Planning:

"Help me plan a project that reads voltage, logs it, and shows it in a Flask dashboard."

GPT will pull answers directly from the uploaded documents customized to your hardware and documents (e.g., EI-52).

Key SUSI Libraries & Their Purpose

Library/Function	Description
<code>SusiLibInitializ()</code>	Must be called first. Sets up access to SUSI functions.

<code>SusiBoardGetValue()</code>	Reads hardware monitoring values (temp, voltage, fan speed, etc.).
<code>SusiGPIOGetLevel()</code> / <code>SetLevel()</code>	Reads/writes GPIO pin status.
<code>SusiWDogStart()</code>	Starts watchdog timer to auto-reset system if software hangs.
<code>SusiI2C*</code> / <code>SusiSMB*</code>	Communicate with onboard I2C or SMBus devices.
<code>SusiVgaSetBacklightBrightness()</code>	Controls screen brightness (if applicable).
<code>SusiIoT*</code> JSON API	Converts data into JSON for IoT dashboards and cloud use.
<code>SusiStorageAreaRead</code> / <code>Write()</code>	Store and retrieve non-volatile data on board flash.

Ask GPT to expand on any function or macro ID (e.g., `SUSI_ID_HWM_TEMP_CPU`).

Real example, compile and test with make example using GPT

GPT can help write a full working example:

Example prompt or question to ask GPT:

"Generate a `Makefile` and `main.c` to get CPU temperature using `SusBoardGetValue`. I want to build this on Linux Ubuntu. Give me step by step instructions, teach me how to make/use the files using `mkdir`, `cd`, `cp`, `nano`, and `touch` using the linux terminal. Define every command and explain what every action does as if I am a person who does not know how to code or use linux."

 1. Create a folder for your SUSI project

bash

 Copy code

```
mkdir susi_temp
```

 What it does: `mkdir` means "make directory." This creates a folder called `susi_temp`.

 2. Go inside the folder

bash

 Copy code

```
cd susi_temp
```

 What it does: `cd` means "change directory." This moves you into the new `susi_temp` folder.

3. Copy the SUSI header files into this folder

bash

 Copy code

```
cp /usr/local/Advantech/SUSI/Susi4Demo/*.h ./
```

 **What it does:** `cp` means "copy". You're copying all the `.h` header files from the demo folder into your current folder (`./` means "here").

4. Make a new file called `main.c` and open it

bash

 Copy code

```
touch main.c  
nano main.c
```

 **What it does:**

- `touch main.c` creates a new file named `main.c`.
- `nano main.c` opens it in a simple text editor.

5. Paste this C code inside `main.c`

```
c Copy Edit

#include <stdio.h>
#include <stdint.h>
#include "Susi4.h"

int main() {
    SusiLibInitialize();
    uint32_t temp;

    if (SusiBoardGetValue(SUSI_ID_HWM_TEMP_CPU, &temp) == 0) {
        printf("CPU Temperature: %.1f °C\n", temp / 10.0);
    } else {
        printf("Failed to read CPU temperature.\n");
    }

    SusiLibUninitialize();
    return 0;
}
```

 What it does:

- Initializes SUSI.
- Reads the CPU temperature using `SusiBoardGetValue`.
- Prints the result.
- Cleans up the library with `SusiLibUninitialize()`.

 Save the file:

- Press `Ctrl + O`, then `Enter`.
- Press `Ctrl + X` to exit Nano.

6. Make a file called `Makefile` and open it

bash

Copy Edit

```
touch Makefile
```

```
nano Makefile
```

What it does:

- `Makefile` tells the system how to compile your C program.

7. Paste this into the `Makefile`

make

Copy Edit

```
TARGET=test
OBSJ=main.o
LIBS=-L/usr/local/Advantech/SUSI/Driver -lsusi-4.00

all:
    gcc -o $(TARGET) $(OBSJ) $(LIBS) -D_LINUX

clean:
    rm -f $(TARGET) *.o
```

What it does:

- `gcc -o test main.o ...` compiles your C file into a program called `test`.
- `-lsusi-4.00` tells it to use the SUSI library.
- `make clean` removes old compiled files if you want to start fresh.

👉 Save and exit (`Ctrl + O`, `Enter`, then `Ctrl + X`).

Step 3: Build & Run the Program

1. Compile your program

bash

 Copy  Edit

```
make
```

 **What it does:** This uses your `Makefile` to build the program.

If it works, you'll see something like:

bash

 Copy  Edit

```
gcc -o test main.o -L/usr/local/Advantech/SUSI/Driver -lsUSI-4.00 -D_LINUX
```

2. Run it with root permission

bash

 Copy  Edit

```
sudo ./test
```

 **What it does:** Runs your program with `sudo` (admin mode), which is required to access SUSI hardware.

 If successful, you'll see something like:

yaml

 Copy  Edit

```
CPU Temperature: 46.9 °C
```



Considerations and additional information

- Use ChatGPT to convert SUSI C outputs into Python-readable formats (e.g., via subprocess).
- Always call `SusiLibUninitialize()` after you're done.
- You can add additional documents as necessary.

- Use GPT to explain, refactor, or extend demo code.
- GPT can make mistakes and the code may not always work.